MYSQL PROJECT

Executive Summary

This project involves a comprehensive analysis of an Ecommerce database using SQL. The objective was to extract actionable business insights regarding sales performance, customer behavior, and operational efficiency. The analysis covers foundational SQL techniques; such as multi-table joins and aggregations, as well as advanced methods including Window Functions (RANK, PERCENT_RANK) and Conditional Logic (CASE statements).

Okoye Angel Ugochi
ugochiangel1205@gmail.com

Q1: Total Sales by Employee

- **Analysis:** This query calculates the total revenue generated by each employee. It joins the Employees, Orders, and OrderDetails tables, applying the discount to the unit price to ensure financial accuracy.
- **Query:**

```
SELECT
    e.EmployeeID,
   CONCAT(e.FirstName, ' ', e.LastName) AS EmployeeName,
    SUM(od.Quantity * od.UnitPrice * (1 - od.Discount)) AS TotalSales
FROM Orders o
JOIN Employees e ON o.EmployeeID = e.EmployeeID
JOIN OrderDetails od ON o.OrderID = od.OrderID
GROUP BY e.EmployeeID, EmployeeName
ORDER BY TotalSales DESC;
```

Result:

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

EmployeeID	EmployeeName	TotalSales
4	Margaret Peacock	9164.996998372022
1	Nancy Davolio	4932.126999069471
3	Janet Leverling	4675.578000145033
2	Andrew Fuller	3489.09499950381
8	Laura Callahan	3448.230999140069
6	Michael Suyama	2902.494000997627
5	Steven Buchanan	2177.7423988413066
9	Anne Dodsworth	2017.9749999155756
7	Robert King	1966.5649994201958

Q2: Top 5 Customers by Sales

- **Analysis:** Identified the most valuable customers by summing their total spending. This helps the marketing team identify candidates for loyalty programs.
- **Query:**

```
SELECT
    c.CustomerName,
    SUM(od.Quantity * od.UnitPrice) AS TotalSpent
FROM Customers c
JOIN Orders o        ON c.CustomerID = o.CustomerID
JOIN OrderDetails od ON o.OrderID   = od.OrderID
GROUP BY c.CustomerID, c.CustomerName
ORDER BY TotalSpent DESC
```

```
LIMIT 5;
```

- **Result:**

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

	CustomerName	TotalSpent
▶	Ernst Handel	3642
	QUICK-Stop	2161
	Frankenversand	1966
	Save-a-lot Markets	1947
	Hungry Owl All-Night Grocers	1744

Q3: Monthly Sales Trend (1997)

- **Analysis:** Grouped sales by month for the year 1997 to identify seasonal trends and high-performing periods.
- **Query:**

```
SELECT
    MONTH(o.OrderDate)              AS Month,
    MONTHNAME(o.OrderDate)          AS MonthName,
    SUM(od.Quantity * od.UnitPrice) AS TotalSales
FROM Orders o
JOIN OrderDetails od ON o.OrderID = od.OrderID
WHERE YEAR(o.OrderDate) = 1997
GROUP BY MONTH(o.OrderDate), MONTHNAME(o.OrderDate)
ORDER BY Month;
```

- **Result:**

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

	Month	MonthName	TotalSales
▶	1	January	6226
	2	February	2091

Q4: Order Fulfillment Time

- **Analysis:** Used a CASE statement to calculate hypothetical fulfillment times (3 days for 1996, 5 days for 1997) to measure employee workload distribution.
- **Query:**

```
SELECT
  e.EmployeeID,
    CONCAT(e.FirstName, ' ', e.LastName) AS EmployeeName,
```

```
    AVG(
        CASE
            WHEN YEAR(o.OrderDate) = 1996 THEN 3
            WHEN YEAR(o.OrderDate) = 1997 THEN 5
        END
    ) AS AvgFulfilmentDays
FROM Orders o
JOIN Employees e ON o.EmployeeID = e.EmployeeID
GROUP BY e.EmployeeID;
```

- **Result:**

Result Grid | Filter Rows: | Export: | Wrap Cell Conten

EmployeeID	EmployeeName	AvgFulfilmentDays
1	Nancy Davolio	3.2069
2	Andrew Fuller	3.4000
3	Janet Leverling	3.8387
4	Margaret Peacock	3.4500
5	Steven Buchanan	3.0000
6	Michael Suyama	3.3333
7	Robert King	3.4286
8	Laura Callahan	3.5926
9	Anne Dodsworth	3.3333

Q5: London Customer Sales

- **Analysis:** Filtered the database specifically for customers located in London to evaluate regional market performance.
- **Query:**

```
SELECT
    c.CustomerName,
    c.City,
    SUM(od.Quantity * od.UnitPrice) AS TotalSales
FROM Customers c
JOIN Orders o        ON c.CustomerID = o.CustomerID
JOIN OrderDetails od ON o.OrderID   = od.OrderID
WHERE c.City = 'London'
GROUP BY c.CustomerID, c.CustomerName, c.City
ORDER BY TotalSales DESC;
```

- **Result:**

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

CustomerName	City	TotalSales
Seven Seas Imports	London	1064
Eastern Connection	London	374
Around the Horn	London	255
Consolidated Holdings	London	108
B"s Beverages	London	75

Q6: Customers with Multiple Orders on the Same Date

- **Analysis:** Used the HAVING clause to identify high-frequency purchasing behavior where a customer placed more than one order in a single day.

- **Query:**

```
SELECT
    c.CustomerName,
    DATE(o.OrderDate) AS OrderDate,
    COUNT(o.OrderID)  AS NumberOfOrders
FROM Customers c
JOIN Orders o ON c.CustomerID = o.CustomerID
GROUP BY c.CustomerID, c.CustomerName, DATE(o.OrderDate)
HAVING COUNT(o.OrderID) > 1
ORDER BY c.CustomerName, OrderDate;
```

- **Result:**

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

CustomerName	City	TotalSales
Seven Seas Imports	London	1064
Eastern Connection	London	374
Around the Horn	London	255
Consolidated Holdings	London	108
B"s Beverages	London	75

Q7: Average Discount per Product

- **Analysis: Calculated the average discount rate offered for each product to understand pricing strategies.**

- **Query:**

```
SELECT
    p.ProductName,
```

```
    ROUND(AVG(od.Discount), 2) AS AvgDiscount
FROM Products p
JOIN OrderDetails od ON p.ProductID = od.ProductID
GROUP BY p.ProductID, p.ProductName
ORDER BY AvgDiscount DESC;
```

- **Result:**

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

ProductName	AvgDiscount
Zaanse koeken	0.09
Røgede sild	0.09
Sirop d'érable	0.08
Tunnbröd	0.08
Spegesild	0.08
Sir Rodney's Marmalade	0.08
Röd Kaviar	0.08
Genen Shouyu	0.07
Mascarpone Fabioli	0.07
Schoggi Schokolade	0.07
Ravioli Angelo	0.07
Rhönbräu Klosterbier	0.07

Q8: Product Order Frequency by Customer

- **Analysis: This query provides a granular view of customer preferences by listing the total quantity of each specific product ordered per customer.**
- **Query:**

```
SELECT
    c.CustomerName,
    p.ProductName,
    SUM(od.Quantity) AS TotalQuantityOrdered
FROM Customers c
JOIN Orders o        ON c.CustomerID = o.CustomerID
JOIN OrderDetails od ON o.OrderID    = od.OrderID
JOIN Products p      ON od.ProductID = p.ProductID
GROUP BY c.CustomerID, c.CustomerName, p.ProductID, p.ProductName
ORDER BY c.CustomerName, TotalQuantityOrdered DESC;
```

- **Result:**

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

CustomerName	ProductName	TotalQuantityOrdered
Ana Trujillo Emparedados y helados	Outback Lager	5
Ana Trujillo Emparedados y helados	Gudbrandsdalsost	1
Antonio Moreno Taquería	Queso Cabrales	24
Around the Horn	Guaraná Fantástica	25
Around the Horn	Ravioli Angelo	25
Around the Horn	Konbu	20
Around the Horn	Gnocchi di nonna Alice	20
Around the Horn	Valkoinen suklaa	15
B"s Beverages	Aniseed Syrup	30
B"s Beverages	Wimmers gute Semmelknödel	9
Berglunds snabbköp	Rhönbräu Klosterbier	30
Berglunds snabbköp	Sir Rodney's Marmalade	28

Resets all sorted columns

Q9: Employee Sales Ranking

- **Analysis: Employed the RANK() window function to create a performance leaderboard, ranking employees from highest to lowest revenue generators.**
- **Query:**

```
SELECT
    CONCAT(e.FirstName, ' ', e.LastName)          AS EmployeeName,
    SUM(od.Quantity * od.UnitPrice)               AS TotalSales,
    RANK() OVER (ORDER BY SUM(od.Quantity * od.UnitPrice) DESC) AS SalesRank
FROM Employees e
JOIN Orders o        ON e.EmployeeID = o.EmployeeID
JOIN OrderDetails od ON o.OrderID    = od.OrderID
GROUP BY e.EmployeeID, e.FirstName, e.LastName
ORDER BY SalesRank;
```

Result:

Result Grid | Filter Rows: | Export: | Wrap Cell Conten

EmployeeName	TotalSales	SalesRank
Margaret Peacock	9642	1
Nancy Davolio	5204	2
Janet Leverling	4918	3
Andrew Fuller	3660	4
Laura Callahan	3641	5
Michael Suyama	3088	6
Steven Buchanan	2296	7
Anne Dodsworth	2100	8
Robert King	2083	9

Q10: Sales by Country and Category

- **Analysis: A multi-level aggregation that shows which product categories perform best in specific geographic regions.**
- **Query:**

```
SELECT
    c.Country,
    cat.CategoryName,
    SUM(od.Quantity * od.UnitPrice) AS TotalSales
FROM Customers c
JOIN Orders o        ON c.CustomerID  = o.CustomerID
JOIN OrderDetails od ON o.OrderID    = od.OrderID
JOIN Products p      ON od.ProductID = p.ProductID
JOIN Categories cat  ON p.CategoryID = cat.CategoryID
GROUP BY c.Country, cat.CategoryID, cat.CategoryName
ORDER BY c.Country, TotalSales DESC;
```

- **Result:**

Result Grid | Filter Rows: | Export: | Wr

Country	CategoryName	TotalSales
Argentina	Produce	36
Argentina	Confections	12
Austria	Dairy Products	756
Austria	Confections	630
Austria	Condiments	579
Austria	Beverages	564
Austria	Produce	509
Austria	Seafood	493
Austria	Grains/Cereals	450
Austria	Meat/Poultry	261
Belgium	Dairy Products	235
Belgium	Meat/Poultry	200

Q11: Year-over-Year Sales Growth

- **Analysis: Used the LAG() function (or self-join logic) to compare current year sales against the previous year to determine growth percentages per product.**
- **Query:**

```
WITH YearlySales AS (
    SELECT
```

```
        p.ProductID,
        p.ProductName,
        YEAR(o.OrderDate)               AS SaleYear,
        SUM(od.Quantity * od.UnitPrice) AS TotalSales
    FROM Products p
    JOIN OrderDetails od ON p.ProductID = od.ProductID
    JOIN Orders o        ON od.OrderID  = o.OrderID
    GROUP BY p.ProductID, p.ProductName, YEAR(o.OrderDate)
)
SELECT
    curr.ProductName,
    curr.SaleYear                     AS CurrentYear,
    curr.TotalSales                   AS CurrentSales,
    prev.TotalSales                   AS PreviousSales,
    ROUND(
        ((curr.TotalSales - prev.TotalSales) / prev.TotalSales) * 100, 2
    )                                 AS GrowthPct
FROM YearlySales curr
JOIN YearlySales prev
    ON curr.ProductID = prev.ProductID
    AND curr.SaleYear = prev.SaleYear + 1
ORDER BY curr.ProductName, curr.SaleYear;
```

- **Result:**

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

ProductName	CurrentYear	CurrentSales	PreviousSales	GrowthPct
Alice Mutton	1997	149	661	-77.46
Aniseed Syrup	1997	100	30	233.33
Boston Crab Meat	1997	102	474	-78.48
Camembert Pierrot	1997	180	972	-81.48
Chais	1997	106	258	-58.91
Chang	1997	365	737	-50.47
Côte de Blaye	1997	149	420	-64.52
Fløtemysost	1997	255	745	-65.77
Geitost	1997	357	343	4.08
Gnocchi di nonna Alice	1997	601	272	120.96
Gorgonzola Telino	1997	70	1204	-94.19
Gravad lax	1997	30	142	-78.87

Q12: Order Quantity Percentile

- **Analysis: Used PERCENT_RANK() to statistically determine where each order falls in terms of size relative to the entire order history.**
- **Query:**

```
SELECT
    o.OrderID,
    SUM(od.Quantity) AS TotalQuantity,
    ROUND(
        PERCENT_RANK() OVER (ORDER BY SUM(od.Quantity)) * 100, 2
    ) AS PercentileRank
FROM Orders o
JOIN OrderDetails od ON o.OrderID = od.OrderID
GROUP BY o.OrderID
ORDER BY PercentileRank DESC;
```

- **Result:**

Result Grid | Filter Rows: | Export: | Wrap Ce

OrderID	TotalQuantity	PercentileRank
10324	241	100
10440	208	99.49
10359	206	98.97
10430	195	98.46
10263	184	97.95
10393	176	97.44
10442	170	96.92
10390	169	96.41
10401	168	95.9
10382	165	95.38
10372	164	94.87
10342	160	94.36

Q13: Products Never Reordered

- **Analysis: Used a HAVING COUNT = 1 filter to identify "one-hit wonder" products that have been sold but never purchased a second time.**
- **Query:**

```
SELECT
    p.ProductName,
    COUNT(od.OrderDetailID) AS TimesOrdered
```

```
FROM Products p
JOIN OrderDetails od ON p.ProductID = od.ProductID
GROUP BY p.ProductID, p.ProductName
HAVING COUNT(od.OrderDetailID) = 1
ORDER BY p.ProductName;
```

- **Result:**

ProductName	TimesOrdered
Chocolade	1
Laughing Lumberjack Lager	1
Mishi Kobe Niku	1
Røgede sild	1

Q14: Most Valuable Product by Revenue

- **Analysis: Applied PARTITION BY within a subquery to isolate the single highest-earning product for every individual category.**
- **Query:**

```
SELECT *
FROM (
    SELECT
        cat.CategoryName,
        p.ProductName,
        SUM(od.Quantity * od.UnitPrice * (1 - od.Discount)) AS Revenue,
        RANK() OVER (PARTITION BY cat.CategoryName ORDER BY SUM(od.Quantity *
od.UnitPrice * (1 - od.Discount)) DESC) AS rnk
    FROM OrderDetails od
    JOIN Products p ON od.ProductID = p.ProductID
    JOIN Categories cat ON p.CategoryID = cat.CategoryID
    GROUP BY cat.CategoryName, p.ProductName
) ranked
WHERE rnk = 1;
```

- **Result:**

CategoryName	ProductName	Revenue	rnk
Beverages	Chang	1060.961000468582	1
Condiments	Louisiana Fiery Hot Pepper Sauce	620.8549999189563	1
Confections	Pavlova	1077.441999600036	1
Dairy Products	Gorgonzola Telino	1206.1330002769828	1
Grains/Cereals	Gnocchi di nonna Alice	827.2089998046868	1
Meat/Poultry	Perth Pasties	824.9239995218813	1
Produce	Longlife Tofu	580.6979997092858	1
Seafood	Nord-Ost Matjeshering	660.0420000255108	1

Q15: Complex Order Details

- **Analysis: A multi-criteria filter to find high-value orders (>$100) that also included a significant discount (>=5%).**
- **Query:**

```
SELECT
    o.OrderID,
    c.CustomerName,
    SUM(od.Quantity * od.UnitPrice)   AS OrderTotal,
    MAX(od.Discount)                  AS MaxDiscount
FROM Orders o
JOIN OrderDetails od ON o.OrderID   = od.OrderID
JOIN Customers c     ON o.CustomerID = c.CustomerID
GROUP BY o.OrderID, c.CustomerName
HAVING
    SUM(od.Quantity * od.UnitPrice) > 100
    AND MAX(od.Discount) >= 0.05
ORDER BY OrderTotal DESC;
```

- **Result:**

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

OrderID	CustomerName	OrderTotal	MaxDiscount
10263	Ernst Handel	744	0.053
10324	Save-a-lot Markets	735	0.082
10359	Seven Seas Imports	734	0.082
10327	Folk och fä HB	640	0.095
10337	Frankenversand	635	0.086
10286	QUICK-Stop	620	0.069
10393	Save-a-lot Markets	587	0.09
10267	Frankenversand	580	0.07
10401	Rattlesnake Canyon Grocery	518	0.088
10344	White Clover Markets	455	0.05
10440	Save-a-lot Markets	445	0.078
10436	Blondel père et fils	440	0.096